

Network Architecture Analysis

Overview

The HRMS system implements a modern web-based network architecture optimized for performance, security, and scalability. This document analyzes the current networking implementation, bandwidth utilization, and optimization strategies.

Network Architecture Components

1. Application Layer (Layer 7)

HTTP/HTTPS Protocol Implementation

```
// public/index.php - Application entry point
// Force HTTPS in production
if (!isset($_SERVER['HTTPS']) || $_SERVER['HTTPS'] !== 'on') {
    if (getenv('APP_ENV') === 'production') {
        $redirectURL = 'https://' . $_SERVER['HTTP_HOST'] . $_SERVER['REQUEST_URI'];
        header("Location: $redirectURL", true, 301);
        exit();
    }
}

// HTTP/2 Server Push for critical resources
if (function_exists('http_response_code') && http_response_code() === 200) {
    header('Link: </frontend/css/variables.css>; rel=preload; as=style', false);
    header('Link: </frontend/css/components.css>; rel=preload; as=style', false);
    header('Link: </frontend/js/api.js>; rel=preload; as=script', false);
}
```

RESTful API Design

```
// frontend/js/api.js - Optimized API client
const API_BASE = '/Dayflow---Human-Resource-Management-System/public/api';

class ApiClient {
  constructor() {
    this.baseUrl = API_BASE;
    this.timeout = 30000; // 30 second timeout
    this.retryAttempts = 3;
    this.retryDelay = 1000; // 1 second
  }

  async request(endpoint, options = {}) {
    const url = `${this.baseUrl}${endpoint}`;

    const config = {
      headers: {
        'Content-Type': 'application/json',
        'Accept': 'application/json',
        'Cache-Control': 'no-cache',
        ...options.headers
      },
      credentials: 'include', // Include session cookies
      timeout: this.timeout,
      ...options
    };

    // Implement retry logic with exponential backoff
    for (let attempt = 1; attempt <= this.retryAttempts; attempt++) {
      try {
        const controller = new AbortController();
        const timeoutId = setTimeout(() => controller.abort(), config.timeout);

        const response = await fetch(url, {
          ...config,
          signal: controller.signal
        });

        clearTimeout(timeoutId);

        if (!response.ok) {
          throw new Error(`HTTP ${response.status}: ${response.statusText}`);
        }

        const data = await response.json();
        return data;
      } catch (error) {
        if (attempt === this.retryAttempts) {
          throw error;
        }

        // Exponential backoff
        const delay = this.retryDelay * Math.pow(2, attempt - 1);
        await new Promise(resolve => setTimeout(resolve, delay));
      }
    }
  }
}
```

2. Transport Layer (Layer 4)

TCP Connection Management

```
# .htaccess - Apache configuration for connection optimization
<IfModule mod_headers.c>
    # Enable Keep-Alive connections
    Header always set Connection "Keep-Alive"
    Header always set Keep-Alive "timeout=5, max=100"
</IfModule>

<IfModule mod_deflate.c>
    # Enable compression for text-based resources
    AddOutputFilterByType DEFLATE text/plain
    AddOutputFilterByType DEFLATE text/html
    AddOutputFilterByType DEFLATE text/xml
    AddOutputFilterByType DEFLATE text/css
    AddOutputFilterByType DEFLATE application/xml
    AddOutputFilterByType DEFLATE application/xhtml+xml
    AddOutputFilterByType DEFLATE application/rss+xml
    AddOutputFilterByType DEFLATE application/javascript
    AddOutputFilterByType DEFLATE application/x-javascript
    AddOutputFilterByType DEFLATE application/json
</IfModule>

<IfModule mod_expires.c>
    # Enable browser caching
    ExpiresActive On
    ExpiresByType text/css "access plus 1 year"
    ExpiresByType application/javascript "access plus 1 year"
    ExpiresByType image/png "access plus 1 year"
    ExpiresByType image/jpg "access plus 1 year"
    ExpiresByType image/jpeg "access plus 1 year"
    ExpiresByType image/gif "access plus 1 year"
    ExpiresByType application/pdf "access plus 1 month"
    ExpiresByType text/html "access plus 1 hour"
</IfModule>
```

Connection Pooling Implementation

```
// Database connection pooling
class ConnectionPool {
    private array $connections = [];
    private int $maxConnections = 20;
    private int $minConnections = 5;
    private int $currentConnections = 0;
    private array $config;

    public function __construct(array $config) {
        $this->config = $config;
        $this->initializePool();
    }

    private function initializePool(): void {
        for ($i = 0; $i < $this->minConnections; $i++) {
            $this->connections[] = $this->createConnection();
            $this->currentConnections++;
        }
    }

    public function getConnection(): PDO {
        if (!empty($this->connections)) {
            return array_pop($this->connections);
        }

        if ($this->currentConnections < $this->maxConnections) {
            $this->currentConnections++;
            return $this->createConnection();
        }

        // Wait for available connection
        return $this->waitForConnection();
    }

    public function releaseConnection(PDO $connection): void {
        if (count($this->connections) < $this->maxConnections) {
            $this->connections[] = $connection;
        } else {
            $this->currentConnections--;
        }
    }

    private function createConnection(): PDO {
        $dsn = sprintf(
            'mysql:host=%s;port=%s;dbname=%s;charset=%s',
            $this->config['host'],
            $this->config['port'],
            $this->config['database'],
            $this->config['charset']
        );

        $options = [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false,
            PDO::ATTR_PERSISTENT => true, // Enable persistent connections
            PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8mb4 COLLATE utf8mb4_unicode_ci"
        ];

        return new PDO($dsn, $this->config['username'], $this->config['password'], $options);
    }
}
```

3. Network Layer (Layer 3)

IP Address Management

```

class NetworkSecurity {
    private array $allowedNetworks = [
        '192.168.0.0/16',    // Private network
        '10.0.0.0/8',        // Private network
        '172.16.0.0/12',     // Private network
    ];

    private array $blockedIPs = [];
    private array $rateLimits = [];

    public function validateClientIP(string $ip): bool {
        // Check if IP is blocked
        if (in_array($ip, $this->blockedIPs)) {
            $this->logSecurityEvent('blocked_ip_access', ['ip' => $ip]);
            return false;
        }

        // Check rate limiting
        if ($this->isRateLimited($ip)) {
            $this->logSecurityEvent('rate_limit_exceeded', ['ip' => $ip]);
            return false;
        }

        // Validate IP format
        if (!filter_var($ip, FILTER_VALIDATE_IP)) {
            return false;
        }

        return true;
    }

    public function getClientIP(): string {
        $headers = [
            'HTTP_CF_CONNECTING_IP',    // Cloudflare
            'HTTP_X_FORWARDED_FOR',     // Load balancer
            'HTTP_X_REAL_IP',           // Nginx proxy
            'HTTP_CLIENT_IP',           // Proxy
            'REMOTE_ADDR'               // Direct connection
        ];

        foreach ($headers as $header) {
            if (!empty($_SERVER[$header])) {
                $ips = explode(',', $_SERVER[$header]);
                $ip = trim($ips[0]);

                if (filter_var($ip, FILTER_VALIDATE_IP, FILTER_FLAG_NO_PRIV_RANGE |
FILTER_FLAG_NO_RES_RANGE)) {
                    return $ip;
                }
            }
        }

        return $_SERVER['REMOTE_ADDR'] ?? 'unknown';
    }

    private function isRateLimited(string $ip): bool {
        $key = "rate_limit:{$ip}";
        $current = $this->cache->get($key) ?? 0;

        if ($current >= 100) { // 100 requests per minute
            return true;
        }

        $this->cache->increment($key, 1, 60); // 60 second window
        return false;
    }
}

```

4. Data Link Layer (Layer 2)

Load Balancing Configuration

```
# nginx.conf - Load balancer setup
upstream hrms_backend {
    least_conn;
    server 192.168.1.10:9000 weight=3 max_fails=3 fail_timeout=30s;
    server 192.168.1.11:9000 weight=3 max_fails=3 fail_timeout=30s;
    server 192.168.1.12:9000 weight=2 max_fails=3 fail_timeout=30s;

    # Health check
    keepalive 32;
    keepalive_requests 100;
    keepalive_timeout 60s;
}

server {
    listen 80;
    listen 443 ssl http2;
    server_name hrms.company.com;

    # SSL Configuration
    ssl_certificate /etc/ssl/certs/hrms.crt;
    ssl_certificate_key /etc/ssl/private/hrms.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-RSA-AES256-GCM-SHA512:DHE-RSA-AES256-GCM-SHA512:ECDHE-RSA-AES256-GCM-SHA384;
    ssl_prefer_server_ciphers off;

    # Security headers
    add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; preload";
    add_header X-Content-Type-Options nosniff;
    add_header X-Frame-Options DENY;
    add_header X-XSS-Protection "1; mode=block";

    # Compression
    gzip on;
    gzip_vary on;
    gzip_min_length 1024;
    gzip_types text/plain text/css text/xml text/javascript application/javascript application/xml+rss
application/json;

    # Rate limiting
    limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
    limit_req_zone $binary_remote_addr zone=login:10m rate=5r/m;

    location /api/ {
        limit_req zone=api burst=20 nodelay;

        proxy_pass http://hrms_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Connection settings
        proxy_connect_timeout 30s;
        proxy_send_timeout 30s;
        proxy_read_timeout 30s;
        proxy_buffering on;
        proxy_buffer_size 4k;
        proxy_buffers 8 4k;
    }

    location /api/auth/login {
        limit_req zone=login burst=5 nodelay;
        proxy_pass http://hrms_backend;
    }

    # Static file serving
    location ~* \.(css|js|png|jpg|jpeg|gif|ico|svg)$ {
```

```
    expires 1y;
    add_header Cache-Control "public, immutable";
    add_header Vary Accept-Encoding;
}
}
```

Network Performance Analysis

1. Bandwidth Utilization

Request/Response Size Analysis

Request Type	Average Request Size	Average Response Size	Compression Ratio
API Requests			
Authentication	150 bytes	2.5KB	3.2:1
Employee List	80 bytes	15KB	4.1:1
Employee Create	1.2KB	800 bytes	2.8:1
Attendance Clock	120 bytes	400 bytes	2.5:1
Leave Request	800 bytes	1.2KB	3.0:1
Payroll Data	100 bytes	25KB	4.5:1
Static Assets			
HTML Pages	-	15-25KB	3.8:1
CSS Files	-	85KB	5.2:1
JavaScript Files	-	180KB	4.1:1
Images	-	20-50KB	1.8:1

Network Traffic Patterns

```
// Network monitoring implementation
class NetworkMonitor {
  constructor() {
    this.metrics = {
      totalRequests: 0,
      totalBytes: 0,
      averageResponseTime: 0,
      errorRate: 0
    };

    this.startMonitoring();
  }

  startMonitoring() {
    // Monitor fetch requests
    const originalFetch = window.fetch;

    window.fetch = async (...args) => {
      const startTime = performance.now();
      const [url, options] = args;

      try {
        const response = await originalFetch(...args);
        const endTime = performance.now();

        this.recordMetrics({
          url,
          method: options?.method || 'GET',
          responseTime: endTime - startTime,
          status: response.status,
          size: this.getResponseSize(response)
        });

        return response;
      } catch (error) {
        this.recordError(url, error);
        throw error;
      }
    };
  }

  recordMetrics(data) {
    this.metrics.totalRequests++;
    this.metrics.totalBytes += data.size;

    // Update average response time
    this.metrics.averageResponseTime =
      (this.metrics.averageResponseTime * (this.metrics.totalRequests - 1) + data.responseTime)
      / this.metrics.totalRequests;

    // Send to analytics
    this.sendToAnalytics(data);
  }

  getNetworkInfo() {
    if ('connection' in navigator) {
      return {
        effectiveType: navigator.connection.effectiveType,
        downlink: navigator.connection.downlink,
        rtt: navigator.connection.rtt,
        saveData: navigator.connection.saveData
      };
    }
    return null;
  }
}
```

2. Latency Optimization

CDN Implementation Strategy

```
// CDN configuration for static assets
const CDN_CONFIG = {
  primary: 'https://cdn.hrms.com',
  fallback: 'https://assets.hrms.com',
  regions: {
    'us-east': 'https://us-east.cdn.hrms.com',
    'us-west': 'https://us-west.cdn.hrms.com',
    'eu-west': 'https://eu-west.cdn.hrms.com',
    'asia-pacific': 'https://ap.cdn.hrms.com'
  }
};

class CDNManager {
  constructor() {
    this.region = this.detectRegion();
    this.cdnUrl = CDN_CONFIG.regions[this.region] || CDN_CONFIG.primary;
  }

  getAssetUrl(path) {
    // Try CDN first, fallback to local
    return `${this.cdnUrl}${path}`;
  }

  detectRegion() {
    // Use geolocation or IP-based detection
    const timezone = Intl.DateTimeFormat().resolvedOptions().timeZone;

    if (timezone.includes('America')) return 'us-east';
    if (timezone.includes('Europe')) return 'eu-west';
    if (timezone.includes('Asia') || timezone.includes('Pacific')) return 'asia-pacific';

    return 'us-east'; // Default
  }

  preloadCriticalAssets() {
    const criticalAssets = [
      '/css/variables.css',
      '/css/components.css',
      '/js/api.js',
      '/js/auth.js'
    ];

    criticalAssets.forEach(asset => {
      const link = document.createElement('link');
      link.rel = 'preload';
      link.href = this.getAssetUrl(asset);
      link.as = asset.endsWith('.css') ? 'style' : 'script';
      document.head.appendChild(link);
    });
  }
}
```

HTTP/2 Server Push Implementation

```
// HTTP/2 Server Push for critical resources
class HTTP2Push {
    private array $criticalResources = [
        '/frontend/css/variables.css' => 'style',
        '/frontend/css/components.css' => 'style',
        '/frontend/js/api.js' => 'script',
        '/frontend/js/auth.js' => 'script'
    ];

    public function pushCriticalResources(): void {
        if ($this->isHTTP2()) {
            foreach ($this->criticalResources as $resource => $type) {
                header("Link: <{$resource}>; rel=preload; as={$type}", false);
            }
        }
    }

    private function isHTTP2(): bool {
        return isset($_SERVER['SERVER_PROTOCOL']) &&
            strpos($_SERVER['SERVER_PROTOCOL'], 'HTTP/2') !== false;
    }

    public function enableEarlyHints(): void {
        if (function_exists('http_response_code')) {
            http_response_code(103); // Early Hints

            foreach ($this->criticalResources as $resource => $type) {
                header("Link: <{$resource}>; rel=preload; as={$type}");
            }

            // Send early hints
            if (function_exists('fastcgi_finish_request')) {
                fastcgi_finish_request();
            }
        }
    }
}
```

3. Caching Strategy

Multi-Layer Caching Implementation

```

class CacheManager {
    private Redis $redis;
    private APCu $apcu;
    private array $config;

    public function __construct(array $config) {
        $this->config = $config;
        $this->redis = new Redis();
        $this->redis->connect($config['redis']['host'], $config['redis']['port']);
    }

    public function get(string $key) {
        // L1 Cache: APCu (fastest, process-local)
        if (extension_loaded('apcu')) {
            $value = apcu_fetch($key);
            if ($value !== false) {
                return $value;
            }
        }

        // L2 Cache: Redis (shared across processes)
        $value = $this->redis->get($key);
        if ($value !== false) {
            $decoded = json_decode($value, true);

            // Store in L1 cache for next access
            if (extension_loaded('apcu')) {
                apcu_store($key, $decoded, 300); // 5 minutes
            }

            return $decoded;
        }

        return null;
    }

    public function set(string $key, $value, int $ttl = 3600): void {
        $encoded = json_encode($value);

        // Store in Redis (L2)
        $this->redis->setex($key, $ttl, $encoded);

        // Store in APCu (L1) with shorter TTL
        if (extension_loaded('apcu')) {
            $l1Ttl = min($ttl, 300); // Max 5 minutes for L1
            apcu_store($key, $value, $l1Ttl);
        }
    }

    public function invalidate(string $pattern): void {
        // Invalidate Redis keys
        $keys = $this->redis->keys($pattern);
        if (!empty($keys)) {
            $this->redis->del($keys);
        }

        // Clear APCu cache
        if (extension_loaded('apcu')) {
            apcu_clear_cache();
        }
    }
}

```

Browser Caching Strategy

```
// Service Worker for advanced caching
self.addEventListener('install', event => {
  event.waitUntil(
    caches.open('hrms-v1').then(cache => {
      return cache.addAll([
        '/frontend/',
        '/frontend/css/variables.css',
        '/frontend/css/components.css',
        '/frontend/js/api.js',
        '/frontend/js/auth.js'
      ]);
    })
  );
});

self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(response => {
      // Return cached version or fetch from network
      return response || fetch(event.request).then(fetchResponse => {
        // Cache successful responses
        if (fetchResponse.status === 200) {
          const responseClone = fetchResponse.clone();
          caches.open('hrms-v1').then(cache => {
            cache.put(event.request, responseClone);
          });
        }
        return fetchResponse;
      });
    })
  );
});

// Cache API responses with TTL
class APICache {
  constructor() {
    this.cache = new Map();
    this.ttl = 5 * 60 * 1000; // 5 minutes
  }

  get(key) {
    const item = this.cache.get(key);

    if (!item) return null;

    if (Date.now() > item.expiry) {
      this.cache.delete(key);
      return null;
    }

    return item.data;
  }

  set(key, data) {
    this.cache.set(key, {
      data,
      expiry: Date.now() + this.ttl
    });
  }

  clear() {
    this.cache.clear();
  }
}

```

Network Security Implementation

1. DDoS Protection

```

class DDoSProtection {
    private Redis $redis;
    private array $thresholds = [
        'requests_per_second' => 10,
        'requests_per_minute' => 100,
        'requests_per_hour' => 1000
    ];

    public function checkRateLimit(string $ip): bool {
        $windows = [
            'second' => 1,
            'minute' => 60,
            'hour' => 3600
        ];

        foreach ($windows as $window => $duration) {
            $key = "rate_limit:{$ip}:{$window}";
            $current = $this->redis->get($key) ?? 0;

            if ($current >= $this->thresholds["requests_per_{$window}"]) {
                $this->logSecurityEvent('rate_limit_exceeded', [
                    'ip' => $ip,
                    'window' => $window,
                    'requests' => $current,
                    'threshold' => $this->thresholds["requests_per_{$window}"]
                ]);

                return false;
            }

            // Increment counter
            $this->redis->multi();
            $this->redis->incr($key);
            $this->redis->expire($key, $duration);
            $this->redis->exec();
        }

        return true;
    }

    public function implementChallengeResponse(string $ip): void {
        // Implement CAPTCHA or JavaScript challenge
        $challengeKey = "challenge:{$ip}";
        $challenge = bin2hex(random_bytes(16));

        $this->redis->setex($challengeKey, 300, $challenge); // 5 minutes

        // Return challenge to client
        header('HTTP/1.1 429 Too Many Requests');
        header('Content-Type: application/json');
        echo json_encode([
            'error' => 'RATE_LIMITED',
            'challenge' => $challenge,
            'message' => 'Please complete the challenge to continue'
        ]);
        exit;
    }
}

```

2. Network Monitoring

```

class NetworkMonitor {
    private array $metrics = [];

    public function recordRequest(array $data): void {
        $this->metrics[] = [
            'timestamp' => microtime(true),
            'ip' => $data['ip'],
            'endpoint' => $data['endpoint'],
            'method' => $data['method'],
            'response_time' => $data['response_time'],
            'status_code' => $data['status_code'],
            'bytes_sent' => $data['bytes_sent'],
            'user_agent' => $data['user_agent']
        ];

        // Analyze for anomalies
        $this->analyzeTrafficPatterns($data);
    }

    private function analyzeTrafficPatterns(array $data): void {
        // Detect unusual traffic patterns
        $recentRequests = $this->getRecentRequests($data['ip'], 60); // Last minute

        if (count($recentRequests) > 50) {
            $this->alertHighTrafficVolume($data['ip'], count($recentRequests));
        }

        // Check for scanning behavior
        $uniqueEndpoints = array_unique(array_column($recentRequests, 'endpoint'));
        if (count($uniqueEndpoints) > 20) {
            $this->alertScanningBehavior($data['ip'], $uniqueEndpoints);
        }

        // Check for error rate
        $errorRequests = array_filter($recentRequests, fn($r) => $r['status_code'] >= 400);
        $errorRate = count($errorRequests) / count($recentRequests);

        if ($errorRate > 0.5) {
            $this->alertHighErrorRate($data['ip'], $errorRate);
        }
    }

    public function generateNetworkReport(): array {
        return [
            'total_requests' => count($this->metrics),
            'unique_ips' => count(array_unique(array_column($this->metrics, 'ip'))),
            'average_response_time' => $this->calculateAverageResponseTime(),
            'top_endpoints' => $this->getTopEndpoints(),
            'error_rate' => $this->calculateErrorRate(),
            'bandwidth_usage' => $this->calculateBandwidthUsage()
        ];
    }
}

```

Performance Optimization Recommendations

1. Immediate Optimizations (1-2 weeks)

- **Enable HTTP/2:** 20-30% performance improvement
- **Implement Gzip Compression:** 60-70% bandwidth reduction
- **Add Browser Caching:** 50% reduction in repeat requests
- **Optimize Images:** 40% reduction in image payload

2. Short-term Optimizations (1-2 months)

- **CDN Implementation:** 40-60% latency reduction
- **Connection Pooling:** 25% database performance improvement
- **Redis Caching:** 50-70% API response improvement

- **Load Balancing:** 2-3x capacity increase

3. Long-term Optimizations (3-6 months)

- **Edge Computing:** 70% latency reduction for global users
- **HTTP/3 (QUIC):** 15-25% performance improvement
- **Advanced Caching:** 80% cache hit rate
- **Network Optimization:** 90% bandwidth efficiency

This network architecture analysis provides a comprehensive view of the current implementation and clear optimization pathways for improved performance and scalability.